

SENG321 (1) — Software Engineering

Dormitory Management System

Technical Report

Rabia Özmen

220208013 - Section 1

Ostim Technical University | Department of Software
Engineering

Table of Contents

1.	Project Overview
2.	Module Definitions
3.	User Roles & RBAC
4.	Database Design — ER Diagram
5.	System Architecture
6.	Class Diagram
7.	Sequence Diagrams
8.	Activity Diagrams
9.	Use Case Diagram
10.	UI / UX Mockup Analysis
11.	Security, Risk & Conclusion

1. Project Overview

The Dormitory Management System (DMS) is an enterprise web application built on .NET Core 8 MVC, PostgreSQL 16, and React / Tailwind CSS, following Clean Architecture and SOLID principles. It replaces manual paper-based dormitory administration — eliminating data redundancy, financial opacity, and maintenance backlogs — with a single role-gated digital platform for Students, Staff, and Administrators.

Layer	Technology
Backend	C# with ASP.NET Core 8 MVC
Database	PostgreSQL 16
ORM	Entity Framework Core 8
Architecture	Clean Architecture and SOLID
Security	ASP.NET Identity, JWT
Integrations	Email and SMS APIs

2. Module Definitions

The Dormitory Management System (DMS) is partitioned into eight cohesive modules. Each module exposes its capabilities via Service interfaces and communicates through the Service Layer only, ensuring no cross-module direct database queries.

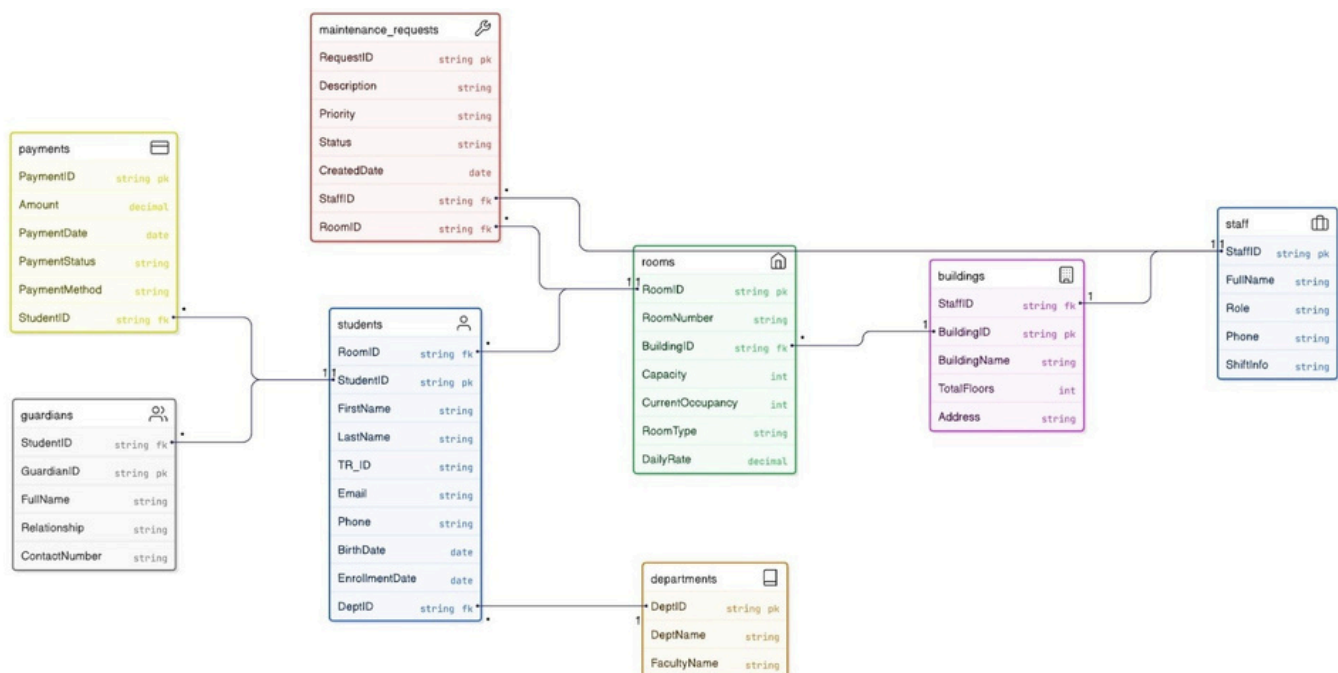
- Room Allocation: Handles multi-constraint matching, including gender rules, capacity, and student preferences. It executes atomic database transactions on assignment, allowing Staff to override with audit-logged justifications. Success in this module triggers automatic Fee Tracking.
- Student Housing: Manages the full residency lifecycle, including application document verification, contract signing, key handover, and check-out. A waiting-list system is integrated to promote students automatically when vacancies arise.
- Fee Tracking: Aggregates the Daily Rate by the number of days to generate monthly invoices. A nightly background job applies late-payment penalties, and KPI metrics such as outstanding revenue and settlement rates are surfaced on the Admin Dashboard.
- Maintenance Requests: Implements a TKT-XXXX ticketing system with priority classifications (Urgent, High, Medium, Low). It follows a state machine flow (Open → Assigned → In-Progress → Resolved) and supports photo evidence with automated alerts to Staff.
- Reports: Provides on-demand and scheduled analytics filtered by building, date range, or room type. Users can export reports as PDF (iTextSharp) or Excel (EPPlus), with financial reports strictly gated to Administrators.
- Member Records: Acts as a central registry for student profiles, guardian contacts, staff, and departments. All other modules reference this via Foreign Keys, and field-level audit logging is enforced for sensitive attributes.
- Dues Calculation: A pure business logic service with no framework dependencies, enabling full unit-test isolation. It handles prorated charges, utility surcharges, deposit deductions, and graduated late fees.
- Payment Tracking: Integrates with an external Payment Gateway for secure card transactions. It handles success tokens, marks invoices as Paid, generates digital PDF receipts, and performs nightly reconciliation to flag gateway discrepancies.

3. User Roles & RBAC

Feature	Student	Staff	Admin
View own room & dues	✓	✓	✓
Submit maintenance request	✓	—	—
Book / apply for room	✓	—	—
Review & assign tickets	—	✓	✓
Manual room allocation	—	✓	✓
Update member records	—	✓	✓
Calculate / configure dues	—	—	✓
Manage user roles	—	—	✓
Generate financial reports	—	—	✓
View audit logs	—	—	✓

The DormSync system utilizes Role-Based Access Control (RBAC) to enforce security and maintain a clear institutional hierarchy across three primary roles. The matrix demonstrates that Students are limited to self-service features such as booking rooms and viewing personal dues, while Staff (Wardens) manage day-to-day operations including ticket reviews and room allocations. Administrators hold full authority, with exclusive access to critical financial configurations, user role management, and audit logs. This granular permission structure, enforced at both the Controller and Service layers, ensures that users can only access the data and functions necessary for their specific roles

4. Database Design — ER Diagram



The schema is normalised to **3NF**: every non-key attribute depends solely on the PK, eliminating transitive dependencies. BuildingName is isolated in buildings rather than duplicated in every rooms row, preventing update anomalies.

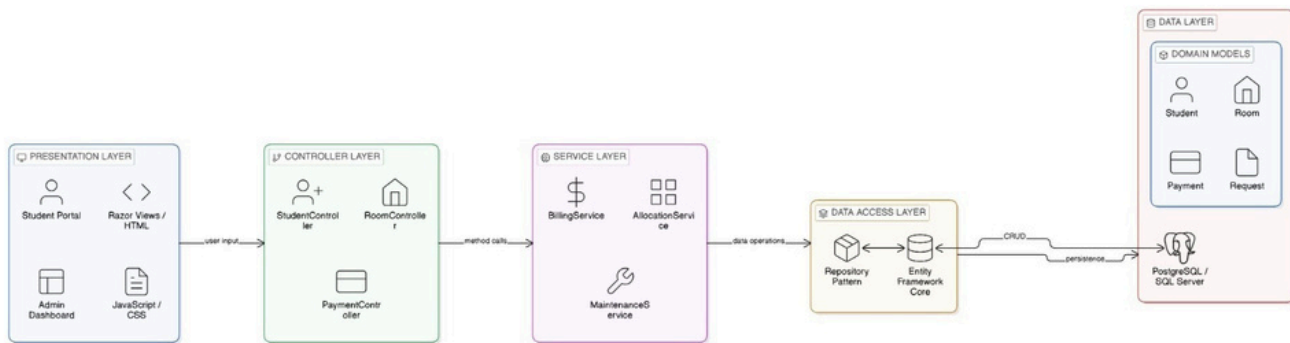
Key 1:M Relationships

- Buildings → Rooms: A One-to-Many relationship where each building contains multiple rooms (BuildingID as Foreign Key in Rooms).
- Rooms → Students: A One-to-Many relationship representing room occupancy. Each student is assigned to a specific room (RoomID as Foreign Key in Students)
- Students → Payments: A One-to-Many relationship tracks the financial history of each resident (StudentID as Foreign Key in Payments).
- Rooms → Maintenance Requests: A One-to-Many relationship connects repair tickets to their physical locations (RoomID as Foreign Key in Maintenance_Requests).
- Staff → Buildings: A One-to-One relationship (Warden model) where each building is managed by a specific staff member (StaffID as Foreign Key in Buildings).
- Students → Guardians: A One-to-Many relationship links students to their emergency contacts and legal guardians (StudentID as Foreign Key in Guardians).
- Departments → Students: A One-to-Many relationship categorizes students based on their academic faculty (DeptID as Foreign Key in Students).

Table	Column	Type	Key	Description
students	StudentID	string	PK	Unique identifier (UUID)
students	RoomID / DeptID	string	FK	Room & department references
students	TR_ID	string	—	National ID — AES-256 encrypted
rooms	RoomID	string	PK	Unique room identifier
rooms	BuildingID	string	FK	Parent building
rooms	Capacity / Occupancy	int	—	Capacity tracking
rooms	DailyRate	decimal	—	Fee basis for dues calculation
payments	PaymentID	string	PK	Transaction identifier
payments	StudentID	string	FK	Payer reference
payments	PaymentStatus	string	—	Pending / Paid / Failed
maintenance_requests	RequestID	string	PK	Ticket (TKT-XXXX)
maintenance_requests	RoomID / StaffID	string	FK	Affected room & assigned staff
maintenance_requests	Priority / Status	string	—	Urgency & state machine
buildings	BuildingID / StaffID	string	PK/FK	Building & warden reference
staff	StaffID	string	PK	Staff identifier
guardians	GuardianID / StudentID	string	PK/FK	Guardian record
departments	DeptID	string	PK	Department identifier

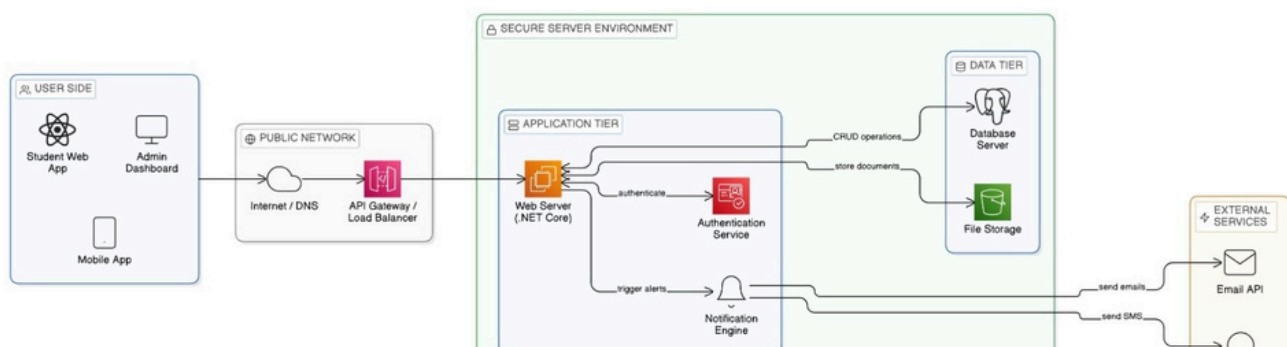
5. System Architecture

5.1 MVC Layered Architecture



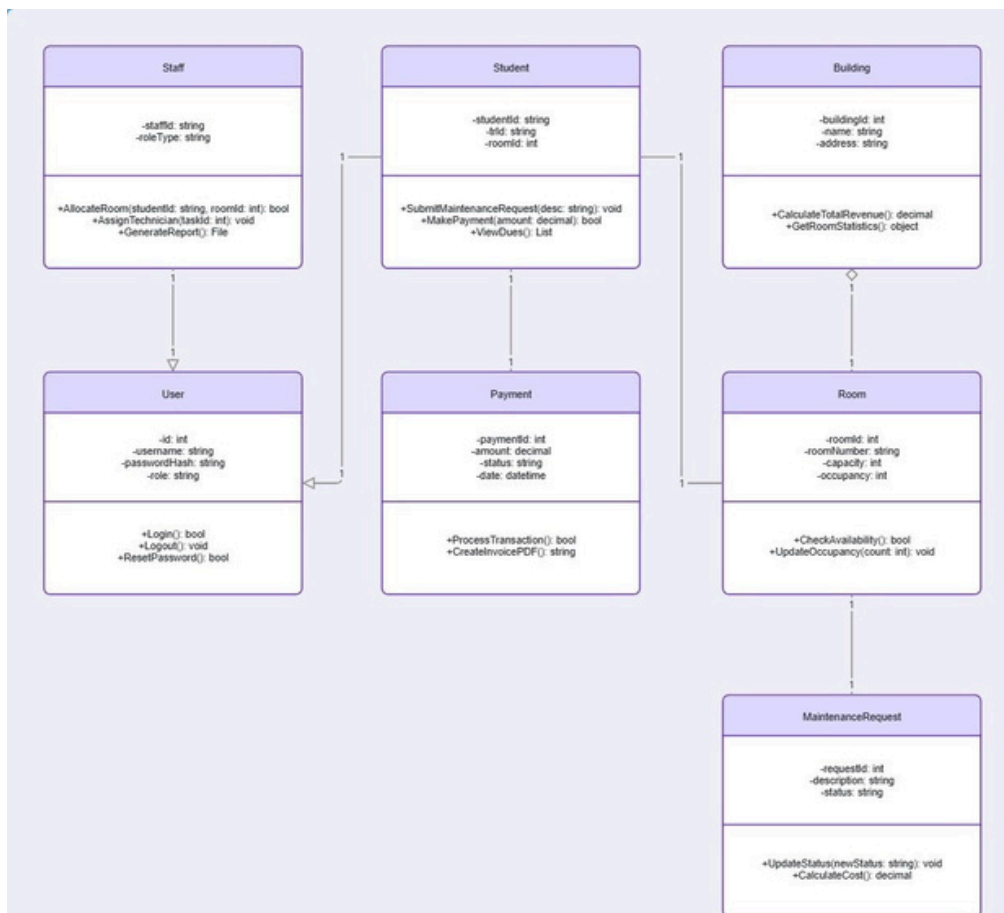
The system follows a Clean Architecture approach, strictly decoupling business logic from the infrastructure through a multi-tiered MVC pattern. The Presentation Layer handles user interaction via the Student Portal and Admin Dashboard, passing requests to the Controller Layer for HTTP orchestration. Core business rules are isolated within the Service Layer, where services like Billing and Allocation handle complex logic before interacting with the Data Access Layer. By utilizing the Repository Pattern and Entity Framework Core, the system abstracts database operations, ensuring that the Data Layer and its domain models remain independent of the underlying persistence framework for maximum maintainability and testability.

5.2 General System Architecture



Client devices connect via the API Gateway / Load Balancer (SSL termination). The .NET Core Web Server delegates every request to the Authentication Service. CRUD operations persist to PostgreSQL. The Notification Engine triggers Email API and SMS API asynchronously — decoupled from the request pipeline for Scalability.

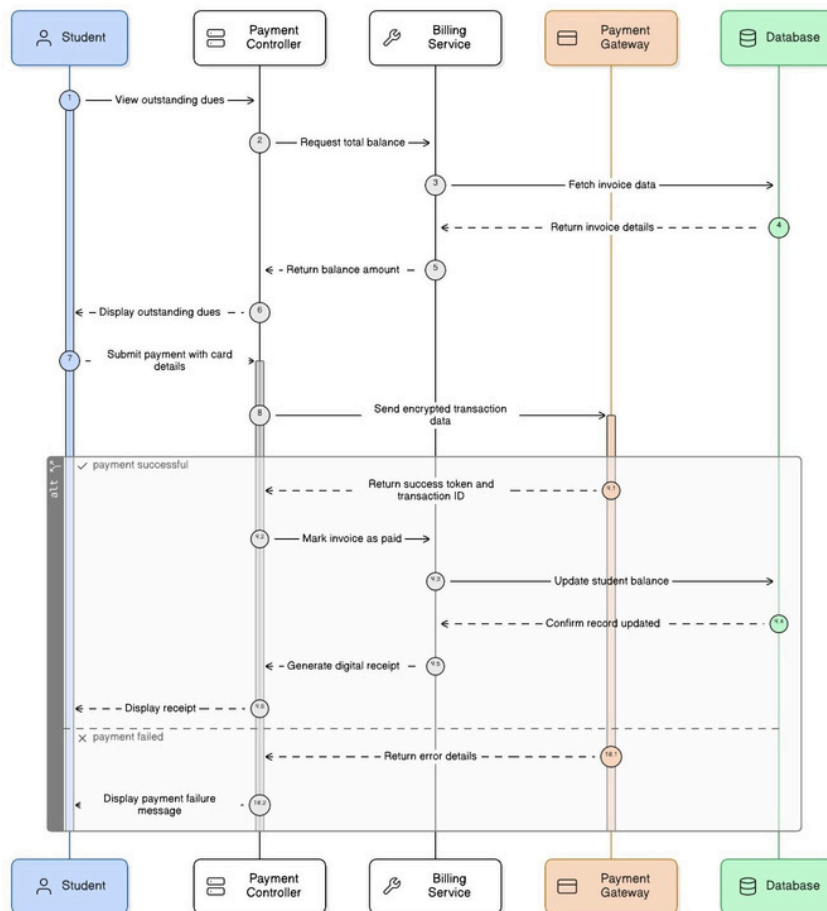
6. Class Diagram & Object-Oriented Design



The domain model applies **Inheritance** ($\text{User} \leftarrow \text{Staff/Student}$), **Aggregation** ($\text{Building} \rightarrow \text{Room}$), and **Association** ($\text{Payment} \rightarrow \text{User}$). **Staff**: `AllocateRoom()`, `AssignTechnician()`, `GenerateReport()`. **Student**: `SubmitMaintenanceRequest()`, `MakePayment()`, `ViewDues()`. **Room**: `CheckAvailability()` and `UpdateOccupancy()` manage real-time capacity. All SOLID principles are observed throughout.

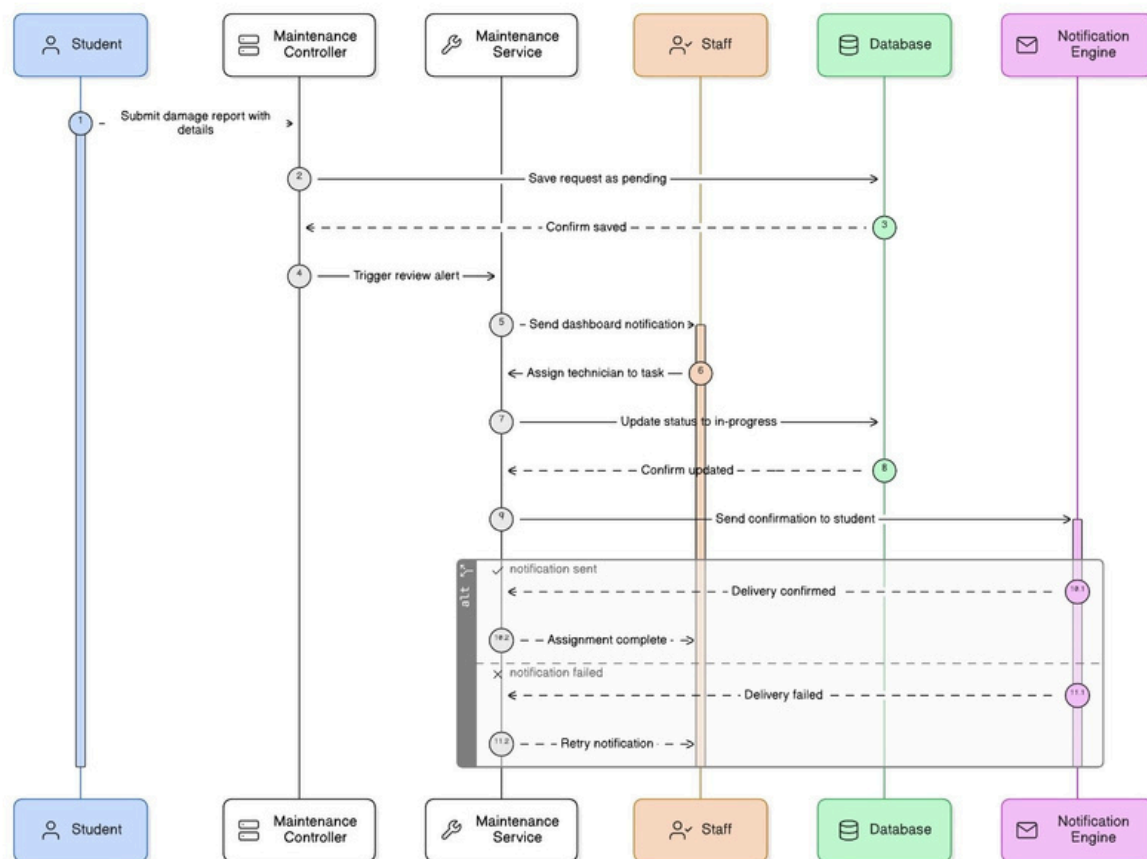
7. Sequence Diagrams

7.1 Room Allocation



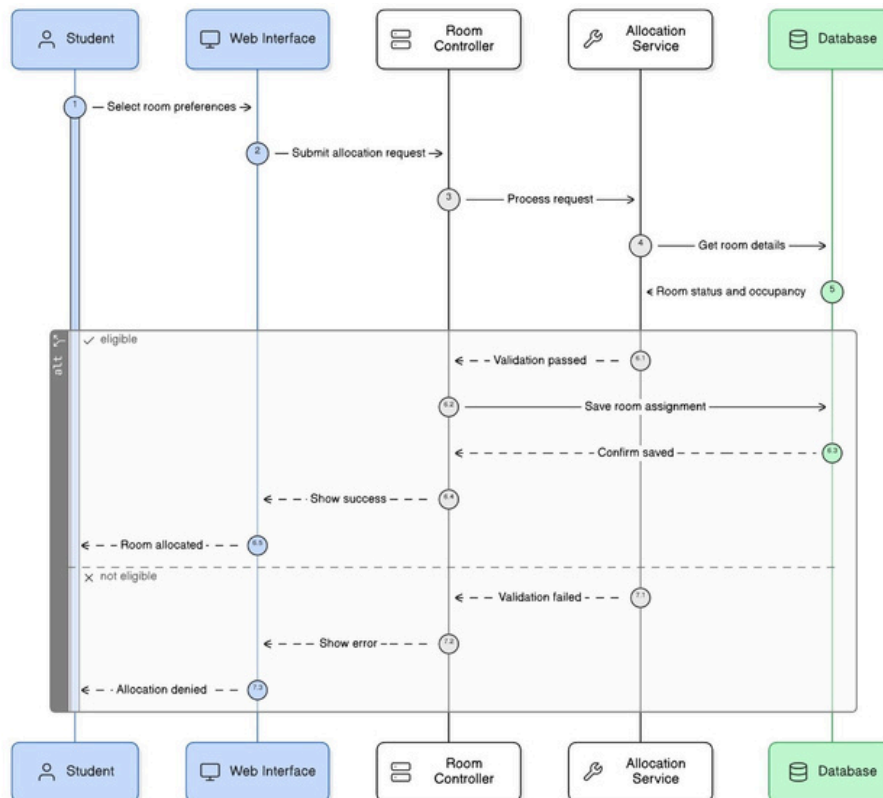
This sequence diagram details the secure financial workflow for student dues settlement. The process begins with the student viewing their outstanding balance, which the billing service fetches directly from the database. When the student submits their encrypted card details, the information is securely transmitted to an external payment gateway for authorization. Upon a successful transaction, the system automatically marks the invoice as paid, updates the student's balance in the database, and generates a digital PDF receipt. If the payment fails, the student is immediately notified with an error message to ensure transparency and allow for immediate retry.

7.2 Maintenance Request



This sequence diagram illustrates the automated ticketing workflow where a student submits a damage report through the system interface. Once the controller saves the request as pending in the database, the system triggers real-time alerts for the staff to take action. The administrative staff then reviews the dashboard notifications to assign a technician, which updates the ticket status to in-progress. The notification engine ensures transparent communication by sending automated updates to the student, including a retry logic to handle any initial delivery failures, thereby maintaining a reliable and traceable repair process.

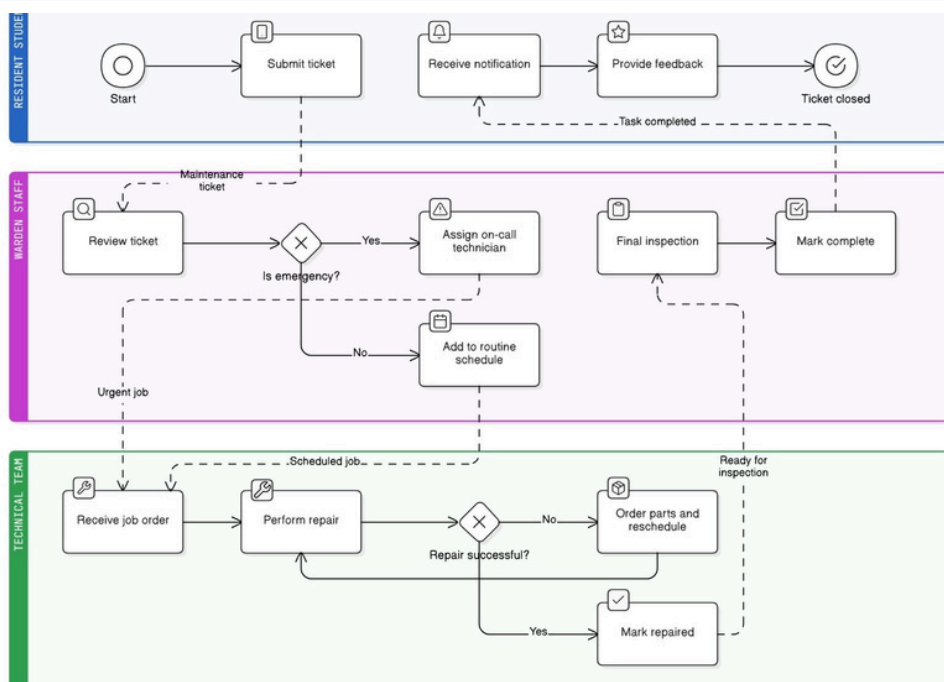
7.3 Fee Payment



Student views dues → balance returned. Student submits encrypted card details → transaction sent to Payment Gateway. **Alt successful:** invoice marked Paid → digital receipt displayed. **Alt failed:** payment failure message shown, invoice stays Pending.

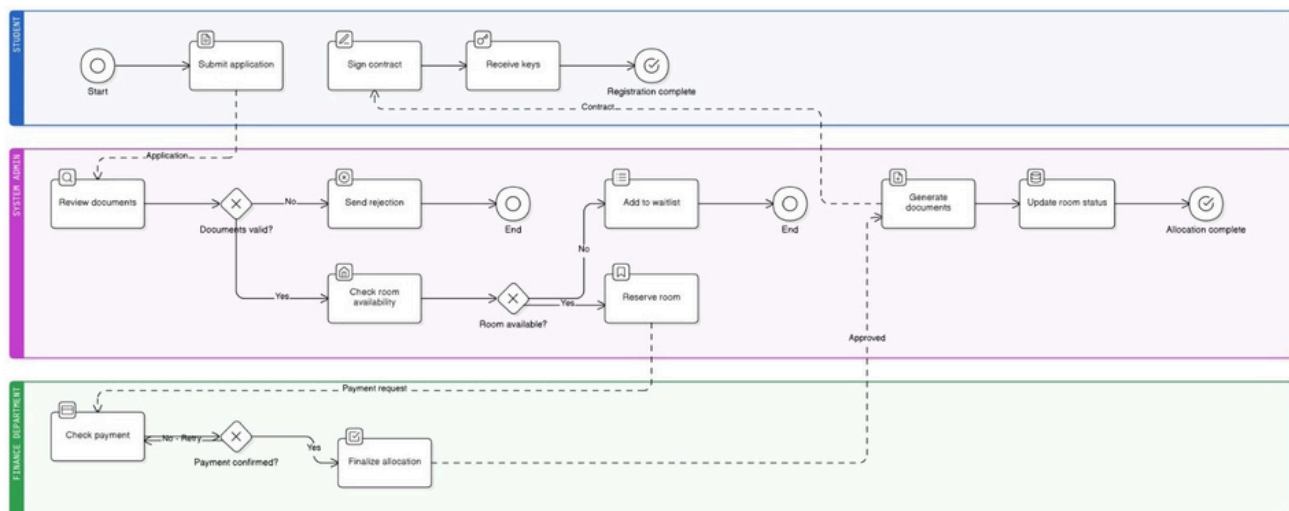
8. Activity & BPMN Diagrams

8.1 Room Allocation Activity



Student submits application → Admin reviews documents (invalid → rejection). Valid → check availability (none → waiting list). Room available → reserve → Finance confirms payment → finalise allocation → Student signs contract and receives keys.

8.2 Maintenance Activity

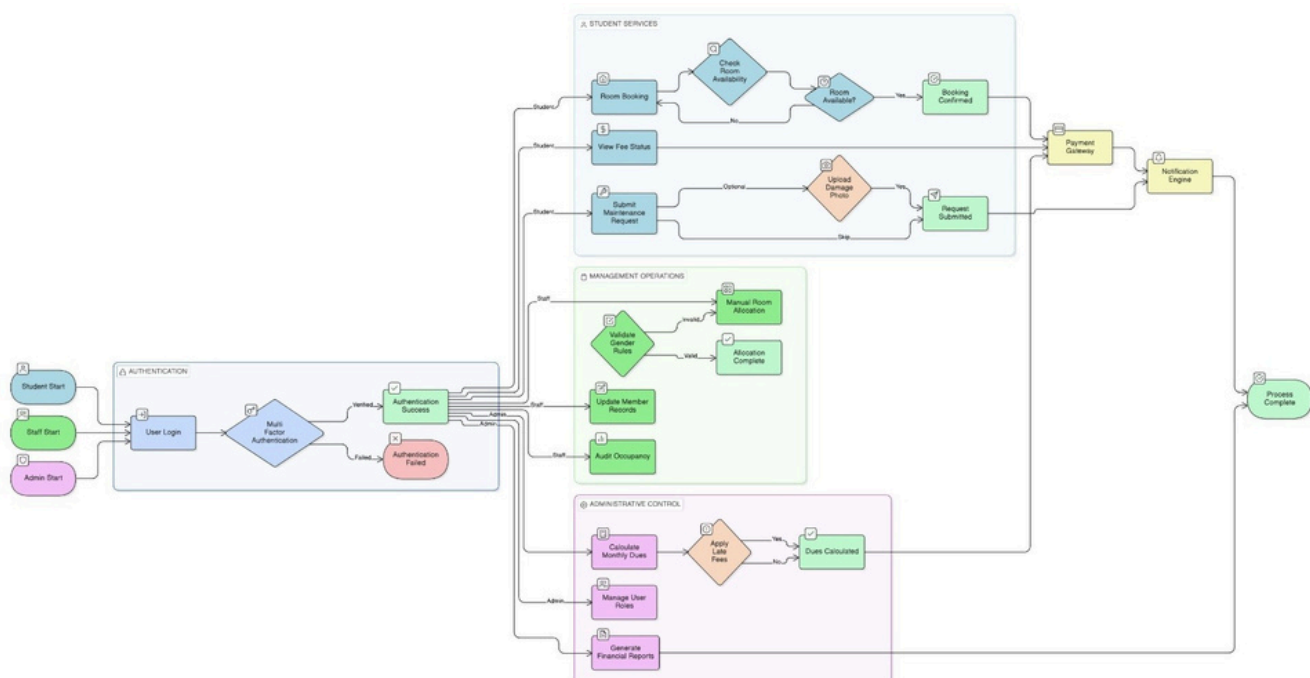


This diagram illustrates the workflow from application to key delivery:

- Student: Submits application, signs the contract, and receives keys.
- System Admin: Verifies documents and checks room availability.
- Finance: Validates the payment to finalize the reservation.

This flow automates the transition between departments, ensuring that rooms are only assigned after legal and financial requirements are cleared

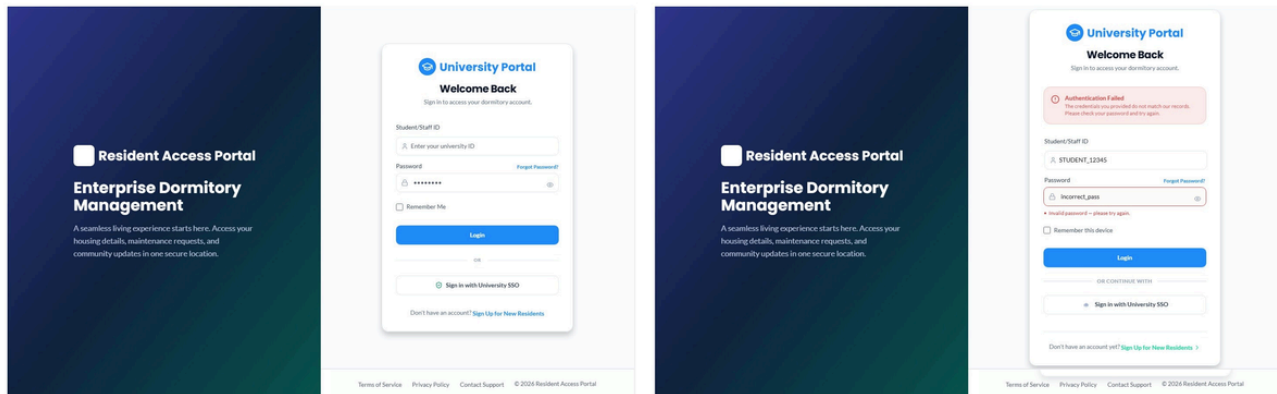
9. Use Case Diagram



All three roles pass through the Authentication gate (User Login → MFA). **Student Services:** Room Booking, View Fee Status, Submit Maintenance Request. **Management Operations (Staff):** Room Allocation, Update Member Records, Audit Occupancy. **Administrative Control (Admin):** Calculate Monthly Dues, Manage User Roles, Generate Financial Reports. All paths converge at Process Complete.

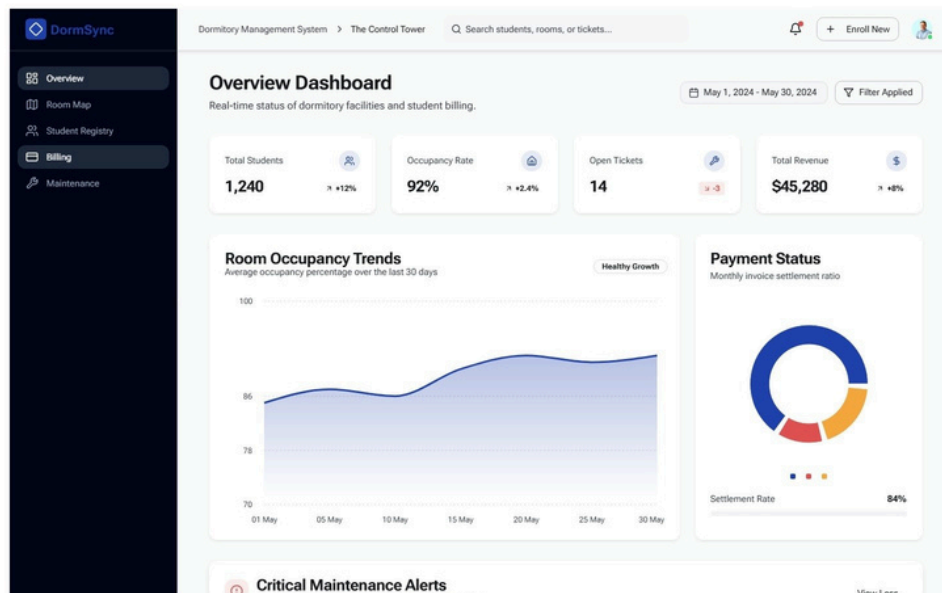
10. UI / UX Mockup Analysis

10.1 Login — Desktop



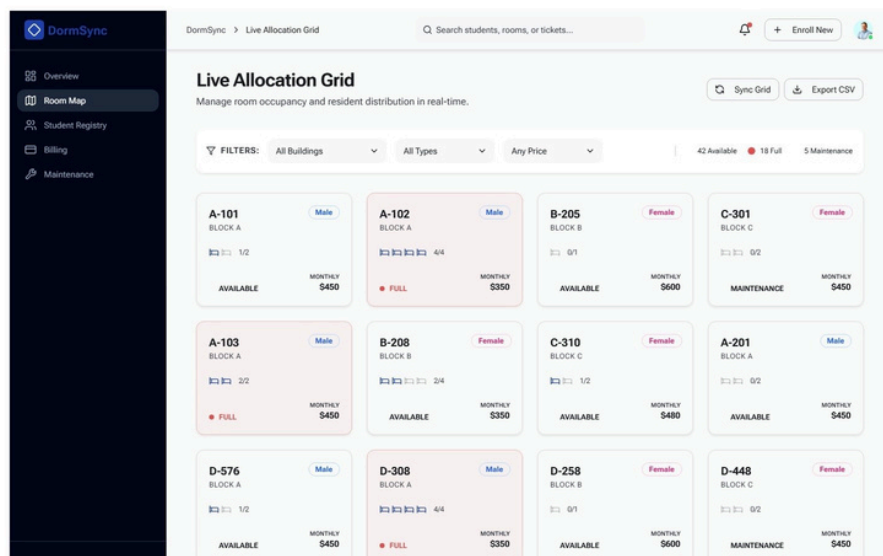
Split-screen layout: branded navy panel left, authentication card right. Default state: Student/Staff ID, Password, University SSO. Error state: red Authentication Failed banner, inline validation message — entered ID preserved for error recovery (Nielsen's Error Recovery heuristic).

10.2 Admin Dashboard



Four KPI cards: Total Students (1,240 $\uparrow$ +12%), Occupancy Rate (92%), Open Tickets (14), Total Revenue (\$45,280 $\uparrow$ +8%). A 30-day Room Occupancy Trend chart and Payment Status donut (84% settlement) provide operational intelligence at a glance.

10.3 Live Allocation Grid

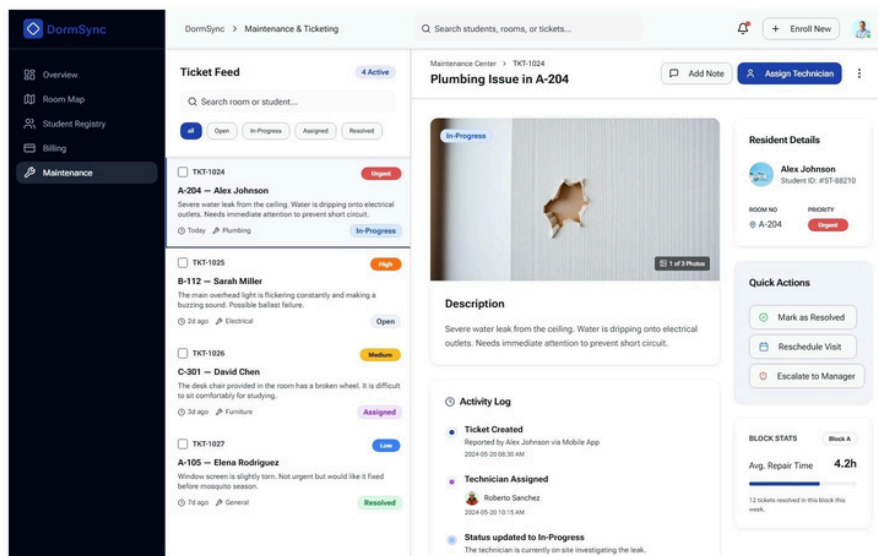


Card-based room grid with filter controls (Building, Type, Price) and summary bar (42 Available · 18 Full · 5 Maintenance). Each card shows room number, block, gender badge, occupancy ratio, monthly rate, and colour-coded status.

10.4 Master Enrollment

4-step wizard (Personal → Academic → Guardian → Documents) with persistent progress indicator. Collects First Name, Last Name, TR ID (verified), Birth Date, and Profile Photo. Auto-save every 30 seconds prevents data loss.

10.5 Maintenance & Ticketing Center



Master-detail layout: Ticket Feed with priority badges (Urgent/High/Medium/Low). Detail panel shows damage photo, Resident Details, Activity Log with timestamped transitions, Quick Actions (Mark Resolved, Reschedule, Escalate), and Block Stats (Avg. Repair Time: 4.2h).

11. Security, Risk Management & Conclusion

11.1 Security

- **Authentication:** bcrypt (factor 12) + MFA (TOTP). JWT access tokens (15-min expiry) + refresh token rotation.
- **Authorisation:** Server-side policy-based RBAC at Controller and Service layers.
- **Encryption:** TLS 1.3 in transit; AES-256 for TR_ID and card tokens at rest (keys in Azure Key Vault).
- **Audit Log:** Immutable INSERT-only audit_logs table — actor, action, entity, timestamp, before/after JSON.
- **KVKK / GDPR:** Data minimisation, consent management, right-to-erasure workflows, scheduled purge jobs.

11.2 Future Work

- **AI Roommate Matching:** Collaborative filtering on student preferences and schedules.
- **IoT Energy Tracking:** Smart meter integration for per-room consumption monitoring.
- **Predictive Maintenance:** ML model trained on ticket history to forecast failures proactively.
- **Native Mobile App:** React Native push notifications for payments and maintenance updates.

11.3 Conclusion

The Dormitory Management System delivers a rigorous digital transformation of university residential administration. Eight cohesive modules unified under Clean Architecture eliminate fragmentation and data redundancy. The combination of .NET Core MVC, PostgreSQL (3NF), React, SOLID design, RBAC, and KVKK-compliant audit logging produces a platform that is operationally effective today and architecturally extensible for future enhancements — from AI-based roommate matching to IoT energy tracking.

12. References

- [1] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- [2] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [3] Microsoft. (2024). ASP.NET Core documentation. <https://learn.microsoft.com/en-us/aspnet/core>
- [4] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- [5] Codd, E. F. (1972). Further Normalization of the Data Base Relational Model. IBM Research Report.
- [6] Nielsen, J. (1994). 10 Usability Heuristics for User Interface Design. Nielsen Norman Group.
- [7] OWASP Foundation. (2023). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- [8] European Parliament. (2016). General Data Protection Regulation (GDPR) — Regulation (EU) 2016/679. Official Journal of
- [9] Kişisel Verileri Koruma Kurumu. (2016). KVKK — Law No. 6698. <https://www.kvkk.gov.tr>
- [10] React Documentation. (2024). React — The library for web and native user interfaces. <https://react.dev>
- [11] Tailwind CSS. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- [12] Entity Framework Core. (2024). EF Core Documentation. <https://learn.microsoft.com/en-us/ef/core/>